

Northwestern University

Weinberg College of Arts and Sciences

Project Environments, Packages, and File Paths

STAT 303-1 · Chapter 2

Lizhen Shi

Continue in the same project folder

```
stat303-setup/  
├─ activity01.ipynb  
├─ activity02.ipynb  
└─ data/  
    └─ station_readings.csv
```

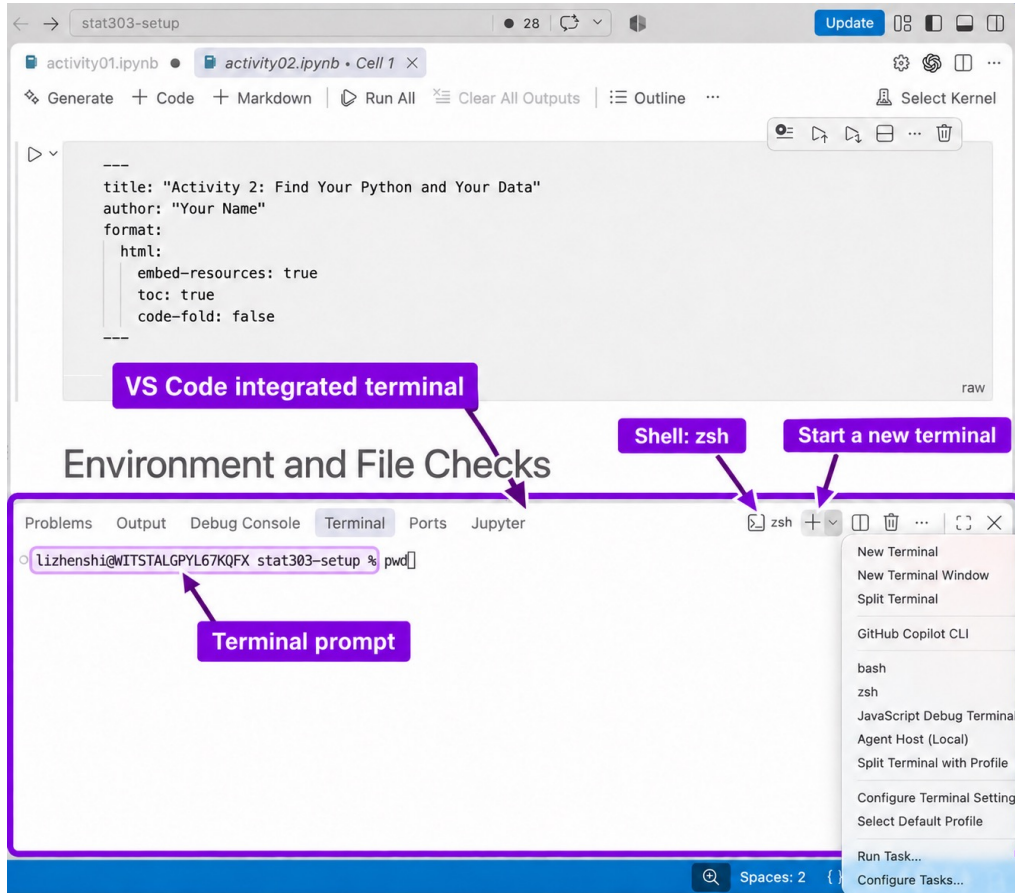
TODAY'S WORK

Navigate folders

Create and verify .venv

Install packages and find data

The terminal and the shell



OPEN IN VS CODE

Terminal → New Terminal

Type a command.

Press Enter.

Read the output.

Wait for the next prompt.

Read the prompt before typing

SHELL PROMPTS

```
student@laptop stat303-setup %
```

```
PS C:\Users\student\stat303-setup>
```

Type after the prompt.
Do not copy it.

INTERACTIVE PYTHON

```
>>>
```

Use `exit()` to return to the shell.

Navigate with the commands for your shell

Action	macOS: zsh / Bash	PowerShell	Command Prompt
Show current folder	pwd	Get-Location	cd
List contents	ls	Get-ChildItem	dir
Enter data	cd data	cd data	cd data
Go to parent	cd ..	cd ..	cd ..

Try it: inspect → enter data → list files → return with cd ..

Ask three questions before a command

01 Where am I typing?

Notebook code cell, shell, or interactive Python?

02 Which Python am I using?

Check `sys.executable` in the notebook and terminal.

03 Which folder is current?

Check the working directory in each place.

Give each project its own environment

PROJECT A

```
Python 3.11  
Shared library: version 1
```

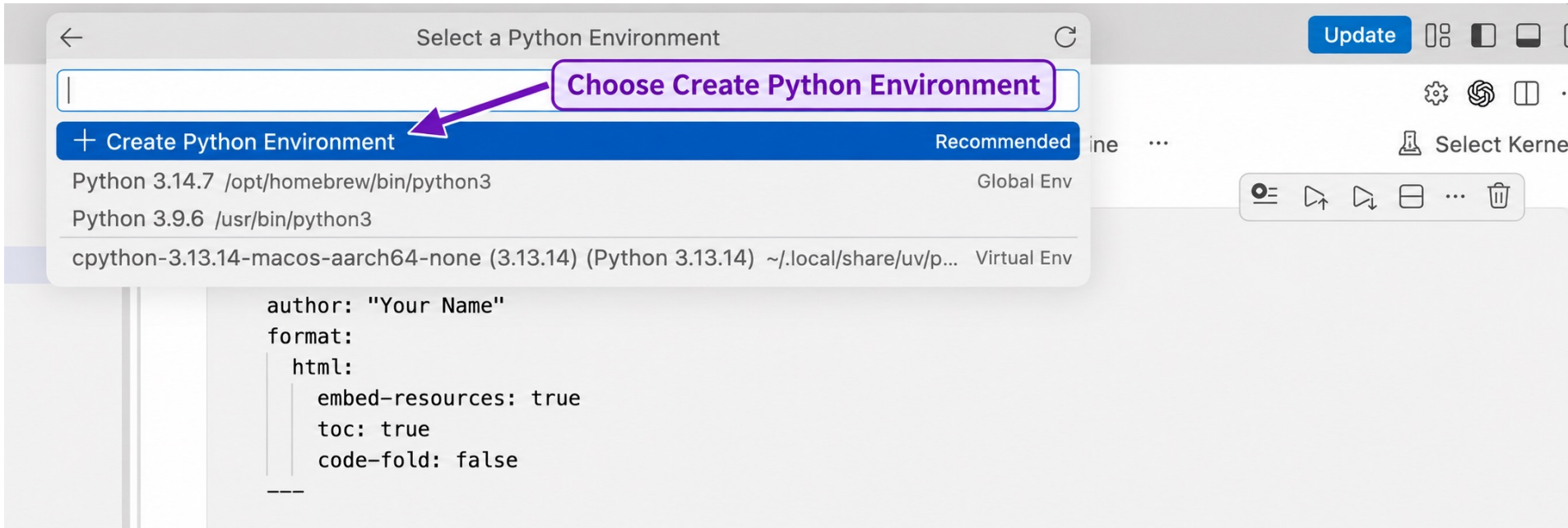
PROJECT B

```
Python 3.12  
Shared library: version 2
```

Each project keeps its own package set.

For STAT303, the practice notebooks share one project .venv.

Create .venv from the notebook



Select Kernel → Python Environments → Create Python Environment

Choose Venv, then the working Python installation from Chapter 1.

Or create `.venv` in the terminal

START IN `stat303-setup`

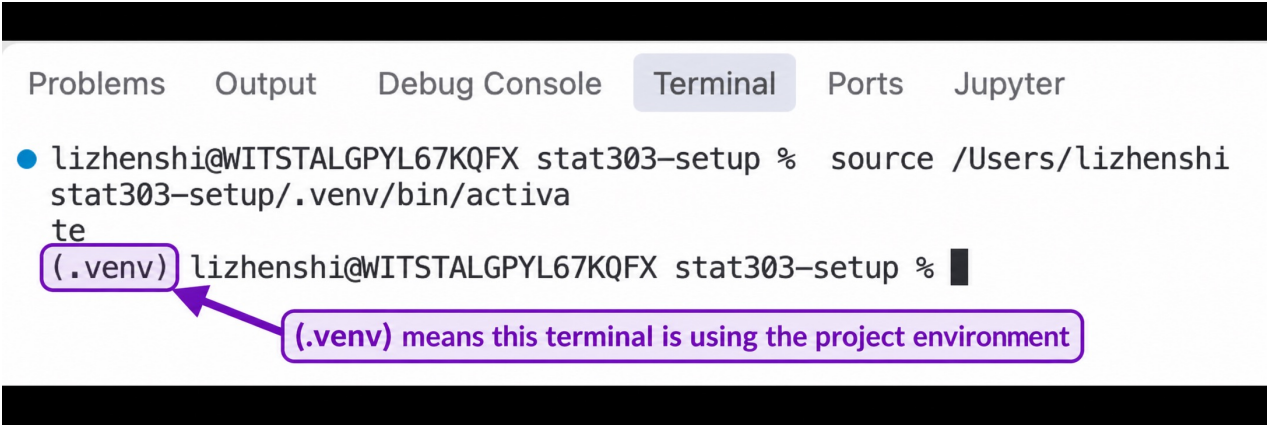
```
python -c "import sys; print(sys.executable) "
```

```
python -m venv .venv
```

Then select the new `.venv` in the notebook kernel picker.

macOS: use `python3` in both commands if `python` is unavailable.

Activate the terminal environment



```
Problems  Output  Debug Console  Terminal  Ports  Jupyter

lizhenshi@WITSTALGPYL67KQFX stat303-setup % source /Users/lizhenshi/stat303-setup/.venv/bin/activate
(.venv) lizhenshi@WITSTALGPYL67KQFX stat303-setup %
```

(.venv) means this terminal is using the project environment

Shell	Activation command
macOS: zsh / Bash	source .venv/bin/activate
Windows PowerShell	.\.venv\Scripts\Activate.ps1
Windows Command Prompt	.venv\Scripts\activate.bat

Verify Python in both places

TERMINAL

```
python -c "import sys; print(sys.executable)"
```

NOTEBOOK CODE CELL

```
import sys  
print(sys.executable)
```

The same project.
The same .venv.

Check: the same Python environment?

TERMINAL

```
/Users/student/stat303-setup/.venv/bin/python
```

NOTEBOOK

```
/Users/student/other-project/.venv/bin/python
```

What would you change before installing packages?

Install packages in the intended environment

TERMINAL · VERIFIED `.venv`

```
pip install ipykernel numpy pandas matplotlib
```

NOTEBOOK · SELECTED `.venv` KERNEL

```
%pip install seaborn
```

Installation puts packages in the environment.

Install once; import in each notebook

```
import numpy as np
import pandas as pd
import matplotlib
import seaborn as sns

print(np.__version__)
print(pd.__version__)
print(matplotlib.__version__)
print(sns.__version__)
```

ON DISK

.venv and packages remain.

IN THE KERNEL SESSION

Rerun imports after restarting.

Record installed packages

macOS / WINDOWS COMMAND PROMPT

```
python -m pip freeze > requirements.txt
```

WINDOWS POWERSHELL

```
python -m pip freeze | Out-File -Encoding utf8 requirements.txt
```

Open requirements.txt and check the entries. Share the Python version too.

Recreate the environment from its record

01 Create and activate a new .venv

Use an installed, compatible Python version.

02 Install the recorded requirements

Run from the folder containing requirements.txt.

```
python -m pip install -r requirements.txt
```

Select the new kernel and repeat the import checks.

Absolute and relative paths

ABSOLUTE · STARTS AT THE FILESYSTEM ROOT

```
/Users/student/stat303-setup/data/station_readings.csv
```

RELATIVE · STARTS AT THE CURRENT WORKING DIRECTORY

```
data/station_readings.csv
```

The relative example expects the current folder to be stat303-setup.

Build a path, then check the file

```
from pathlib import Path

print(Path.cwd())

data_path = Path("data") / "station_readings.csv"
print("Resolved path:", data_path.resolve())
print("File found:", data_path.is_file())
```

`resolve()` shows the location. `is_file()` checks for a file.

Preview the supplied CSV

NOTEBOOK CODE CELL

```
print(data_path.read_text(encoding="utf-8").splitlines()[:3])
```

EXPECTED OUTPUT FOR THIS FILE

```
['station_id,temperature_c', 'S01,18', 'S02,20']
```

These are text lines. A later chapter will load tables with Pandas.

A different working directory changes the path

```
example-project/  
├─ notebooks/  
│   └─ analysis.ipynb  
└─ data/  
    └─ station_readings.csv
```

If the current directory is
example-project/notebooks,
go up one level first.

```
Path("../") / "data" / "station_readings.csv"
```

Check: the terminal moved into data

The notebook's current directory is stat303-setup.
In the terminal, you run `cd data`.

PREDICT BEFORE RUNNING

```
# In the notebook  
print(Path.cwd())
```

Will the notebook's
current folder change?

When `is_file()` returns False

01 Check the current directory

Read `Path.cwd()` and the resolved path.

02 Check the name and layout

Confirm capitalization, extraction, and the data folder.

03 Correct the path using the folder relationship

Keep a separate `wrong_path` for the deliberate failure test.

When you return to the project

01 Open stat303-setup

Select its .venv as the notebook kernel.

02 Check the terminal

Confirm the folder and activate .venv if needed.

03 Run the notebook from the top

Restore imports and variables; repeat the data-path check.

Activity 2: collect the evidence

Part	Evidence to retain in activity02.ipynb
A · Tools and starting folder	Name, shell, explanation, terminal folder and file listing
B · Environment and packages	Matching executable paths; installation and version checks
C · Locations	Terminal navigation beside notebook directory checks
D · Data paths	Predictions, resolved path, True / False checks, CSV preview
E · Dependencies	Export command, two package entries, fresh-run note

Keep explanations in Markdown and label copied terminal output.

Render, inspect, and submit activity02.html

Restart kernel → Run all cells → Save

```
quarto render activity02.ipynb --to html
```

Inspect the HTML and a copy opened outside the project folder.

SUBMIT THE HTML IN THE CHAPTER 2 CANVAS QUIZ

Northwestern University

Before the next command

Where am I typing?

Which Python am I using?

Which folder is current?

Next: Assignment B — Setting Up a Reproducible Python Project